

Assessment Técnico — Sistema de Gestión de Proyectos y Tareas

Java + Spring Boot | Arquitectura Hexagonal

Objetivo

Desarrollar una aplicación en **Java 17+ con Spring Boot 3** que permita gestionar proyectos y tareas, aplicando:

- Clean Architecture (Domain, Application, Infrastructure, Presentation)
- Enfoque Hexagonal (Ports & Adapters)
- Persistencia en base de datos relacional
- Pruebas unitarias automatizadas
- Frontend funcional
- Docker y Docker Compose

El objetivo es evaluar la capacidad del desarrollador para construir una solución profesional, enfocada en reglas de negocio, arquitectura y empleabilidad real.

Modelo de Datos

User

- id (UUID)
- username
- email
- password

Project

- id (UUID)
- ownerId (UUID)
- name



- status (DRAFT, ACTIVE)
- deleted (boolean)

Task

- id (UUID)
- projectId (UUID)
- title
- completed (boolean)
- deleted (boolean)

Relaciones:

- Un proyecto pertenece a un usuario
- Una tarea pertenece a un proyecto

Autenticación y Seguridad Avanzada

Stack

- Spring Security + JWT (access token)

Endpoints

- POST /api/auth/register
- POST /api/auth/login

Reglas de Seguridad

- Endpoints de escritura requieren JWT válido
- Un usuario solo puede modificar sus propios proyectos y tareas
- Accesos indebidos deben retornar 403 Forbidden
- Manejo correcto de 401 Unauthorized y 403 Forbidden

Arquitectura — Clean Architecture (Hexagonal)

La solución debe implementar **Clean Architecture con enfoque Hexagonal (Ports & Adapters)**.

Reglas obligatorias

- El dominio no depende de Spring, JPA ni frameworks
- Las dependencias apuntan siempre hacia el dominio
- Los casos de uso representan acciones del negocio
- Controllers, JPA y seguridad son adaptadores
- No debe existir lógica de negocio en controllers

Puertos y Casos de Uso

Puertos de Entrada (IN) — Casos de uso

Puertos de Entrada (IN) — Casos de Uso

- CreateProjectUseCase
- ActivateProjectUseCase
- CreateTaskUseCase
- CompleteTaskUseCase

Puertos de Salida (OUT) — Necesidades del negocio

Persistencia

- ProjectRepositoryPort
- TaskRepositoryPort

Auditoría

```
public interface AuditLogPort {
```



```
void register(String action, UUID entityId);
}
```

Notificación

```
public interface NotificationPort {
    void notify(String message);
}
```

Usuario Autenticado

```
public interface CurrentUserPort { UUID getCurrentUserId(); }
```

API REST

Proyectos

- POST /api/projects
- GET /api/projects
- PATCH /api/projects/{id}/activate

Tareas

- POST /api/projects/{projectId}/tasks
- PATCH /api/tasks/{id}/complete

Documentación de la API (Swagger / OpenAPI)

La aplicación debe exponer la documentación de la API utilizando **Swagger / OpenAPI**.

Requisitos:

- Configurar Swagger usando **Springdoc OpenAPI**
- Documentar los endpoints REST expuestos por la aplicación
- Permitir el uso de **JWT** desde la interfaz de Swagger para probar endpoints protegidos

La documentación debe estar disponible en tiempo de ejecución en:



- /swagger-ui.html
o
- /swagger-ui/index.html

La configuración de Swagger debe ubicarse en la capa de **infraestructura/configuración**, sin dependencias en el dominio ni en los casos de uso.

Reglas de Negocio

1. Un proyecto solo puede activarse si tiene al menos una tarea activa
2. Solo el propietario puede modificar un proyecto o sus tareas
3. Una tarea completada no puede modificarse
4. Todas las eliminaciones son lógicas (soft delete)
5. La activación de proyectos y la finalización de tareas deben generar auditoría
6. La activación de proyectos y la finalización de tareas deben generar notificación

Pruebas Unitarias Automatizadas

Se deben implementar **al menos 5 pruebas unitarias automatizadas** usando:

- JUnit 5
- Mockito

Pruebas mínimas requeridas

- ActivateProject_WithTasks_ShouldSucceed
- ActivateProject_WithoutTasks_ShouldFail
- ActivateProject_ByNonOwner_ShouldFail
- CompleteTask_AlreadyCompleted_ShouldFail
- CompleteTask_ShouldGenerateAuditAndNotification

Alcance

- Las pruebas deben enfocarse en los **casos de uso críticos del negocio**
- No se espera cobertura total de todos los casos de uso

Ubicación y Enfoque

Las pruebas deben ubicarse en la **capa de aplicación**. Las clases bajo prueba deben ser:

- ActivateProjectUseCase
- CompleteTaskUseCase
- No se deben probar controllers ni adaptadores de infraestructura

Condiciones

- Las pruebas deben ejecutarse sin levantar el contexto de Spring
- Todos los puertos de salida deben ser mockeados

Base de Datos

- PostgreSQL o MySQL
- Spring Data JPA como adaptador
- Script SQL simple o generación automática de esquema
- Datos básicos para pruebas manuales

Docker y Docker Compose

Dockerfile

- Java 17
- Build del backend
- Exposición del puerto de la aplicación

docker-compose.yml

- Backend
- Base de datos

La aplicación debe poder ejecutarse con:

`docker compose up`

Frontend (Obligatorio)

El candidato debe implementar un frontend simple que consuma la API.



Tecnologías permitidas

- Thymeleaf
- React
- Angular
- Vue
- HTML + JavaScript

Funcionalidades mínimas

- Listar proyectos
- Crear tareas
- Completar tareas

No se evalúa diseño visual, únicamente integración y funcionalidad.

Entregables

Repositorio Git con:

- Backend completo
- Frontend funcional
- Arquitectura hexagonal aplicada
- Dockerfile
- docker-compose.yml
- README.md con:
 - ☐ Pasos para ejecutar la aplicación
 - ☐ Credenciales de prueba
 - ☐ Breve descripción de decisiones técnicas

Importante

Es preferible una solución **simple, clara y completamente funcional**



que una solución compleja e incompleta.